

OmniAI

Feature Guide

Everything you need to know -- explained simply.

Milestones 18 * 19 * 20

M18	UX & Accessibility	Bulk actions, exports, dark mode, keyboard shortcuts
M19	Advanced Retrieval	HyDE search, multiple vector databases, streaming, tool calling
M20	Agent Platform	Parallel AI, pause & approve, time-travel, marketplace, sandboxed code

What's In This Guide

M18 -- UX & Accessibility

1. Bulk Document Operations
2. Conversation Export
3. Agent Run Export

M19 -- Advanced Retrieval

4. HyDE Query Expansion
5. pgvector / Pinecone / Weaviate Adapters
6. Streaming Search (Real-time Results)
7. Tool / Function Calling Retrieve
8. Multi-Modal Embeddings (Text + Images)
9. Conversation Forking

M20 -- Agent Platform

10. Parallel Fan-Out / Join
11. Human-in-the-Loop (Pause & Approve)
12. Time-Travel Replay
13. Agent Template Marketplace
14. Multi-Language Code Nodes
15. gVisor Sandbox (Extra-Safe Code Execution)
16. Run Cost Tracking & Alerting

Quick Reference

All API Endpoints

Environment Variables

Built-in Templates

M18 -- UX & Accessibility

1. Bulk Document Operations

What is it? Instead of deleting or updating one document at a time, you can do it to *many* documents all at once -- like selecting lots of files on your desktop and pressing Delete.

How to use it in the app:

- Open the **Knowledge** page.
- Tick the checkboxes next to the documents you want to change.
- A blue bar appears at the top -- choose **Delete**, **Re-index**, or **Set Tags**.

API (for developers):

```
POST /v1/documents/bulk
{
  "document_ids": ["doc_abc", "doc_def"],
  "action": "delete" // or: reindex | set_tags
}
```

The three actions:

delete -- permanently removes the documents

reindex -- re-reads the file and updates the search index

set_tags -- attaches labels (like 'finance', '2025') to help you find them later

2. Conversation Export

What is it? Save any chat conversation to your computer as a neat document -- like taking a screenshot of a chat but as a real file you can open in Word or Notion.

- Open the **Chat** page and select a conversation.
- Click **Download MD** (Markdown -- great for notes) or **Download JSON** (for developers).
- The file downloads automatically.

```
GET /v1/conversations/{id}/export?format=markdown
```

Two formats:

Markdown (.md) -- Looks like a nicely formatted document. Open in Notion, VSCode, or any text editor.

JSON (.json) -- Raw data, useful if you want to process it with code.

3. Agent Run Export

What is it? An Agent is like an AI robot that does a job (searches your documents, runs calculations, writes answers). You can export a record of exactly what the robot did -- step by step -- so you can review, share, or archive it.

- Go to the **Agents** page and open an agent.
- Find a run in the 'Latest output' panel.
- Click **Download MD** or **Download JSON**.

```
GET /v1/agents/{id}/runs/{run_id}/export?format=json
```

M19 -- Advanced Retrieval

4. HyDE Query Expansion

What is it? Normally when you search, the app looks for documents that match your words. HyDE is smarter -- before searching, it asks the AI to *imagine* what a perfect answer looks like, then searches for documents that match *that answer*. It's like asking a librarian to describe the ideal book, then finding it.

Example: You type 'What causes inflation?' Without HyDE the app searches for those exact words. With HyDE the AI first writes a short answer like 'Inflation is caused by too much money chasing too few goods, supply shocks and central bank policy...' then searches for documents about those ideas. Much better results!

How to turn it on (API):

```
POST /v1/retrieve
{
  "query": "What causes inflation?",
  "hyde": true,
  "hyde_model": "gpt-4o"
}
```

If you don't have an AI model set up, it quietly falls back to normal search -- no errors.

5. pgvector / Pinecone / Weaviate Adapters

What is it? OmniAI stores your documents as 'vectors' (numbers that describe the meaning of text). You can choose *where* these vectors are kept. Think of it as choosing which filing cabinet to use.

SEARCH_KIND	What it is	Best for
memory	Stored in RAM (lost on restart)	Testing only
pgvector	PostgreSQL with the pgvector extension	Self-hosted servers

pinecone	Pinecone cloud service	Easy cloud setup
weaviate	Weaviate vector database	Advanced search

How to switch -- add one line to your .env file:

```
SEARCH_KIND=pgvector
DATABASE_URL=postgresql://user:pass@localhost/omniai
```

6. Streaming Search (Real-time Results)

What is it? Normally the app waits until ALL results are found before showing them. Streaming shows results *as they arrive* -- one by one instantly. Think of it like watching a video load progressively vs waiting for the whole file.

```
POST /v1/retrieve/stream
{ "query": "What is quantum computing?", "top_k": 10 }

// Each result arrives as a Server-Sent Event:
data: { "text": "Quantum computers use qubits...", "score": 0.94 }
data: { "text": "Unlike classical bits...", "score": 0.91 }
data: [DONE]
```

7. Tool / Function Calling Retrieve

What is it? You can give AI models (like GPT-4) a special ability to search your documents *automatically* while answering a question. The AI decides when to search and what to search for -- you just ask a question and it does the rest.

```
POST /v1/retrieve/tool
{ "question": "Summarise our Q4 2025 financials", "top_k": 8 }

// Response:
{ "answer": "Q4 2025 revenue was...", "hits": [...], "tool_calls_made": 1 }
```

8. Multi-Modal Embeddings (Text + Images)

What is it? Normally you can only search text. With multi-modal embeddings you can also search using **images**. Send a photo and find documents that are visually or conceptually similar. Powered by the CLIP AI model.

How it works automatically: Any input starting with 'data:image/' is automatically routed to the CLIP image model. Everything else goes to your normal text model. No configuration needed!

Install: pip install open-clip-torch torch Pillow

9. Conversation Forking

What is it? Like saving a game checkpoint. You can 'fork' (branch off) a chat conversation at any point and try a different direction without losing the original. Perfect for exploring 'what if I had asked this differently?'

```
POST /v1/conversations/{id}/fork
{
  "title": "Alternative approach",
  "fork_at_message_id": "msg_abc123" // optional: fork from a specific message
}
```

M20 -- Agent Platform

What is an Agent? An Agent is like a little AI robot with a workflow. You draw a diagram (called a 'graph') that tells it what steps to follow -- search documents, run code, ask a human for approval, then write an answer. M20 makes agents much more powerful with new capabilities.

10. Parallel Fan-Out / Join (Do Multiple Things at Once)

What is it? Normally an agent does one thing at a time. With fan-out it can do **multiple searches simultaneously** -- like having several assistants look up different things at the same time, then combining all their findings.

Node type	What it does
fan_out	Splits the workflow into multiple parallel paths
retrieval	(one per branch -- each can search a different document collection)
join	Waits for ALL branches to finish, then combines all results

Real world example: You ask 'What are the legal and financial risks of this contract?' Fan-out searches *legal documents* AND *financial reports* at the same time. Join merges both sets of results. Your answer covers both topics -- twice as fast!

11. Human-in-the-Loop (Pause & Approve)

What is it? Sometimes you want a **human to check the AI's work** before it proceeds -- like a teacher reviewing a student's answer before it gets sent out. The agent pauses, waits for your approval, then continues.

How it works -- step by step:

- Add a **human_input** node anywhere in your agent's workflow.
- When the agent reaches that node, its status changes to **PAUSED**.
- Your app polls the run status and shows the context to a human reviewer.
- The human approves (or provides corrected text) via the resume endpoint.
- The agent continues from the next node automatically.

Resume a paused run:

```
POST /v1/agents/{agent_id}/runs/{run_id}/resume
{
  "human_input": "Looks correct, please proceed.",
  "approved": true
}
```

Perfect for: Compliance workflows where a manager must approve before an AI-generated email or report is sent. Medical or legal contexts where accuracy is critical.

12. Time-Travel Replay (Debug from Any Point)

What is it? Every agent run is recorded like a video -- every step is logged. You can 'rewind' to any moment and **replay the run from that point** with different inputs. Like using Ctrl+Z -- but for AI workflows.

When is this useful?

- An agent gave a wrong answer -- rewind to just before 'Generate' and try different context.
- You want to test 'what if' scenarios without re-running the full expensive pipeline.
- Debugging a step that failed -- skip directly to that step and re-run it.

How to use it:

```
POST /v1/agents/{agent_id}/runs/{run_id}/replay
{
  "from_event": 3,
  "input_override": "Try this different question instead"
}

// Returns a brand-new run that:
// - fast-forwards through events 0-2 (from the stored recording)
// - then continues LIVE from event 3 with your new input
```

13. Agent Template Marketplace

What is it? Ready-made agent blueprints you can use instantly -- like downloading a free template from the internet and editing it to fit your needs. No need to build from scratch.

Template ID	Name	What it does
basic-rag	Basic RAG	Search then answer. The simplest setup.

parallel-research	Parallel Research	Search two knowledge bases at the same time.
human-review	Human-in-the-Loop	Pause for a human to approve before answering.
code-executor	Code Executor	Search context then run a Python script.
summarisation	Document Summarisation	Find 10 passages and write a summary.

Import a template in one API call:

```
POST /v1/agents/import-template
{
  "template_id": "parallel-research",
  "name": "My Research Agent"
}
```

Or import from your own URL (share templates with your team):

```
POST /v1/agents/import-template
{ "url": "https://your-server.com/my-template.json" }
```

14. Multi-Language Code Nodes

What is it? Agent 'code nodes' can now run **Python, JavaScript, or Bash** -- not just Python. Each language automatically gets the user's question and retrieved context as variables.

Language	Variables available	Example use
Python	user_input, context_text	Data analysis, maths, file processing
JavaScript	user_input, context_text	Web logic, JSON manipulation
Bash	\$USER_INPUT, \$CONTEXT_TEXT	Shell commands, file operations

```
// JavaScript code node example:
const wordCount = context_text.split(' ').length;
console.log(`Found ${wordCount} words in the retrieved context.`);
```

15. gVisor Sandbox (Extra-Safe Code Execution)

What is it? When agents run code, there's a risk it could do something dangerous (like read secret files). A sandbox is like a 'glass box' -- the code runs inside it but cannot touch anything outside. gVisor is the strongest option.

SANDBOX_KIND	Safety level	Requirements
none	No sandbox -- code nodes disabled	Nothing
subprocess	Basic -- runs in a separate process	Nothing extra
docker	Good -- Docker container isolation	Docker installed
gvisor	Best -- intercepts all system calls	gVisor (runsc) installed

Set it in your .env file:


```
SANDBOX_KIND=gvisor
GVISOR_RUNSC_BIN=/usr/local/bin/runsc
```

No gVisor? No problem. If the runsc binary isn't found, OmniAI automatically falls back to 'subprocess' mode and logs a warning. You won't get an error -- it just uses the next-best option silently.

16. Run Cost Tracking & Alerting

What is it? Every agent run now shows you how much it cost (estimated in USD, based on how many AI tokens were used). You can also set a budget limit -- if any single run costs more than that, you get a warning in the logs.

Set a cost alert in your .env:

```
AGENT_RUN_COST_ALERT_USD=0.10 # warn if a run costs more than $0.10
```

See the cost when checking a run:

```
GET /v1/agents/{id}/runs/{run_id}

{ "cost_usd": 0.0042, "status": "COMPLETED", ... }
```

Quick Reference

All New API Endpoints

Method + Path	What it does
POST /v1/documents/bulk	Delete / re-index / tag many documents at once
GET /v1/conversations/{id}/export	Download a chat as JSON or Markdown
POST /v1/conversations/{id}/fork	Branch a conversation from a checkpoint
GET /v1/agents/{id}/runs/{run_id}/export	Download an agent run as JSON or Markdown
POST /v1/retrieve	Search documents (add hyde:true for HyDE)
POST /v1/retrieve/stream	Streaming search -- results appear one by one
POST /v1/retrieve/tool	AI searches your docs automatically while answering
GET /v1/marketplace/templates	Browse ready-made agent blueprints
GET /v1/marketplace/templates/{id}	See the full blueprint for one template
POST /v1/agents/import-template	Create an agent from a blueprint or URL
POST /v1/agents/{id}/runs/{run_id}/resume	Approve and continue a paused (HITL) run
POST /v1/agents/{id}/runs/{run_id}/replay	Time-travel: re-run from any event in history

Key Environment Variables

Variable	What it does	Default
SEARCH_KIND	Which vector database to use	memory
SANDBOX_KIND	How to run agent code safely	none
AGENT_RUN_COST_ALERT_USD	Alert when a run exceeds this cost (\$)	0 (off)
AGENT_MARKETPLACE_URL	Custom template registry URL	(built-in)
GVISOR_RUNSC_BIN	Path to gVisor runsc binary	runsc
PINECONE_API_KEY	Your Pinecone API key	--
WEAVIATE_URL	URL of your Weaviate server	localhost:8080

Getting Started in 3 Steps

Step 1: Log in with your admin account (default: admin@omniai.local)

Step 2: Upload documents to a Collection in the Knowledge page

Step 3: Go to Agents, click 'Import Template', choose 'basic-rag', give it a name, and run it!
Ask it a question about your uploaded documents.

Test Coverage

339 automated tests cover all features described in this guide -- all green. To run them:

```
cd backend
python -m pytest --tb=short -q
```